

Proyecto #3 Control de registro para Parqueo

Ervin Gabriel Gomez García 231226

Bryan Samuel Morales Paredes 23283

INFORMACIÓN Y ANÁLISIS

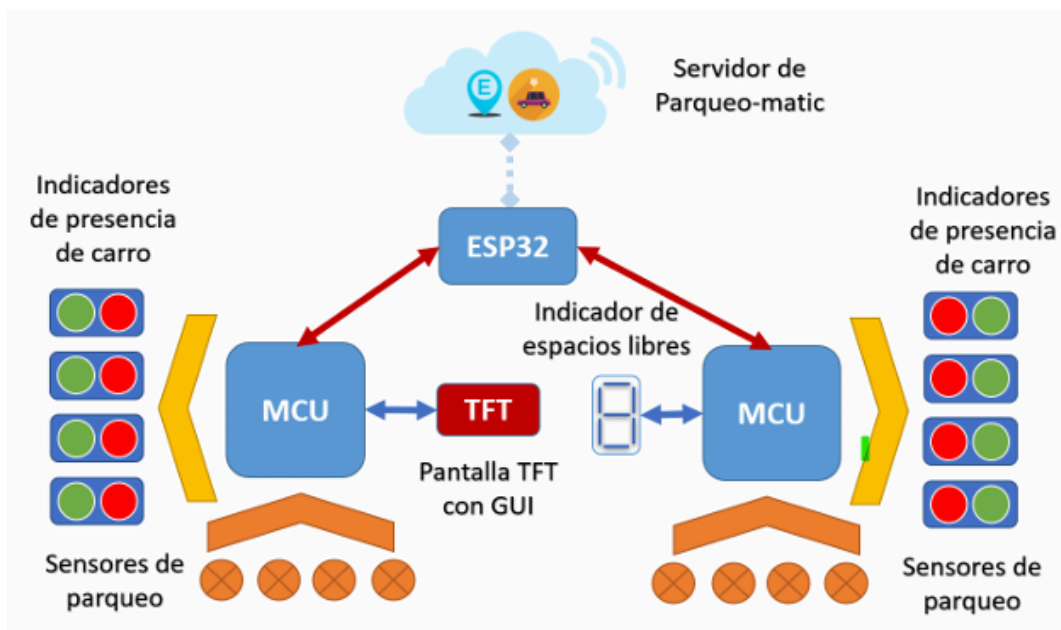


Imagen no.1: Diagrama del sistema del parqueo

En este proyecto se estructura la lógica para el manejo de un parqueo de 2 niveles repartido en 4 parqueos por nivel y un helipuerto en nivel superior, la distribución se ve la imagen 1, el control de los mismo se reparte en dos microcontroladores STM32F446RE los cuales cuentan con el control cuatro parqueos, y uno de ellos con el helipuerto, su control va desde identificar si el parqueo se encuentra disponible identificandolo mediante sensores de luz las cuales son implementadas mediante fotoresistencias con la lógica de un divisor de voltaje, en la lógica de cada uno identifica cual está siendo ocupado de los espacios para luego activar los neopixels correspondientes, y de esta manera actualizar la pantalla que muestra todas las actividades del establecimiento, al igual que un display principal que le muestra al usuario del parqueo si existen parqueo disponibles antes de su ingreso. Además el sistema cuenta con una actualización por WebServer, es el puente principal de intercambio de información entre ambos microcontroladores de cada nivel, mediante el protocolo de comunicación I2C, este mismo muestra mediante una página web los estados de los parqueos si es que se quiere tener un control de manera remota de los parqueos.

LÓGICA JUEGO

La lógica se divide entre las dos STM32F446RE y el ESP32, primero está la lógica de STM32F446RE que tiene la pantalla, en el se tiene la inicialización tanto de las variables, de la pantalla, la carga del fondo y los dibujos de los semáforos iniciales, al igual que el estado de los parqueos se muestran todos en 0, se setea la tira del neopixel y se inicializa la comunicación I2C. En

el loop principal se verifica por un ciclo for que entrada ADC se encuentra activa y si se encuentra en el rango entre 1600 para hacer la activación de que se está ocupando o está ocupado el parqueo, en caso de que el valor sea mayor 1800 significa que el parqueo está desocupado, en el primer rango valida que esté libre para luego decir que está ocupado, de esta manera ejecutar la animación de la función AnimarAutoParqueo en la cual se establece que frame se va a ejecutar, la dirección donde se estará animando el sprite, el sprite y el tamaño del mismo, al inicio de la función se establece la velocidad en que se animará el auto al entrar y salir, se anima desde la posición inicial a la desea de entrada mientras se borra el rastro que deja el autor, luego dibujo el semáforo, en este caso en rojo identificar que el parqueo está ocupado, luego se aumenta la variable cant1 para dar seguimiento de la disponibilidad de los espacios, se pasa ese valor a la función parqueos_disponiblesA para que este muestre en la pantalla cuantos parqueos quedan disponibles en el nivel correspondiente. En caso contrario que el valor ahora esté en un rango mayor a 1800 este ejecuta la segunda secuencia, tomando los valores de la animación de entrada la animación retira el auto desde la posición final en donde se encuentra hasta sacarlo de la pantalla, pone la variable park1A en 0 para decir que está disponible y así poner iniciar otra secuencia, dibuja el semáforo como verde, disminuye la variable cant1 para cargarlo a la función parqueos_disponiblesA para actualizar la cantidad de parqueos disponibles en la pantalla, por último se llama a la función actualizar_leds para activar el neopixel correcto del parque que corresponde, esta logia es la misma para cada una de las entradas ADC y la misma para los valores que son enviadas desde I2C por parte del maestro, pero en este caso solo ve si cada index del buffer de aRxBuffer es igual a 1, para hacer la lógica de ocupación y en caso contrario se hace la secuencia de que el parqueo está disponible

En la lógica de la STM32F446RE que tiene el display, funciona de la misma manera en caso de la lectura de las entradas ADC, en este caso hace la lectura el canal correspondiente para luego enviarlo por I2C al maestro, luego hace la actualización de los neopixeles mediante la variable i que funciona como índice para saber que valor de ADC está activo y así activar el neopixel correcto, por último mostrar la cantidad de parqueos en el display que es enviada desde el maestro mediante I2C. En el caso del maestro que es el ESP32 este mediante el librería Wire espera los valores de los parqueos tanto de A y B que son los esclavos, en este mismo código hace la matemática para saber cuantos parqueos están disponibles, para luego enviarlos a los esclavos y realizar su respectiva lógica, al mismo tiempo mediante el WebServer integrado en el código, muestra en tiempo real y de manera remota mediante conexión a internet en un página las actualizaciones de los parqueos, y mediante el poder dar acceso al parqueo por medio de una talanquera.

VIDEO DE YOUTUBE

<https://youtu.be/89oJOkzh8-c>

LINK REPOSITORIO GITHUB

<https://github.com/Ervin117/Proyecto3-Parqueo.git>

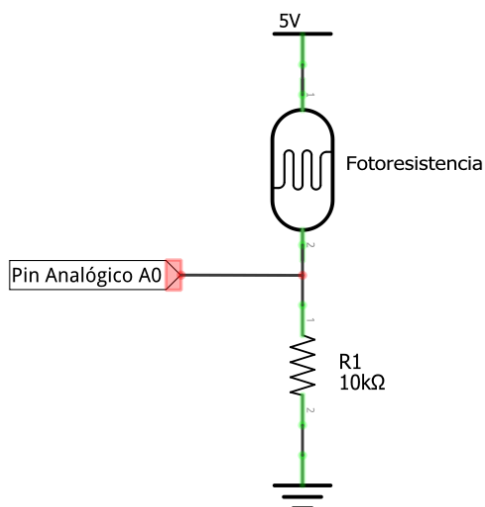


Imagen no.2: Circuito del sensor de luz

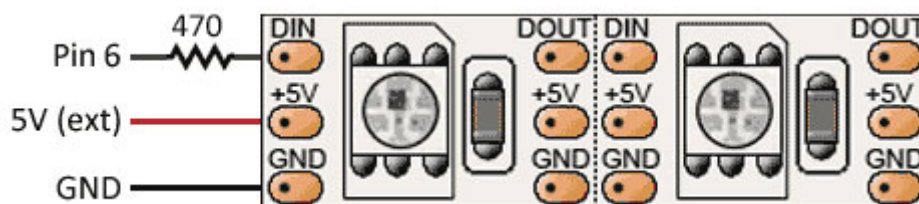


Imagen no.3: Conexión de neopixel